Helm security self assessment (as implemented)

This page describes what the code in this repository does today, and which test shows it. It is a self assessment by the team that built the system. No independent penetration test has been done. Every row below names a file and a test, so a reader can check it.

1 Permission tiers

Tier	Who holds it	Where it is enforced	Tests
T0 Read	All agents, for the inputs of their own step	Agents get typed inputs and a frozen `RulesView` (`helm/rules_view.py`) and import no database code	`test_rt8_agent_modules_have_no_database_access`, `test_rt8_agents_only_see_a_frozen_copy_of_the_rules`
T1 Propose	Reader, Forecaster, Collections, Scheduler, Treasury, Explainer	They return typed records. The Reader has no tools and its output is schema checked with extra fields rejected (`helm/agents/reader.py`)	`test_rt2_reader_rejects_extra_fields_and_has_no_tools`, `test_stub_an_obedient_model_cannot_smuggle_extra_fields`
T2 Decide	Policy Engine, plain code	`helm/engine/policy.py`, pure functions. Rule order is kill switch, statutory first, protected payee, duplicate or anomaly, approval limit, mandate, chance constraint	`tests/test_policy_engine.py`, `tests/test_mandate_auth.py`
T3 Execute	Executor, plain code	`helm/executor.py` sends only ALLOW decisions, rereads the kill switch before every instruction, and is the only caller of `mint_key`	`test_tier_executor_refuses_every_verdict_except_allow`, `test_tier_only_the_executor_mints_bank_keys`, `test_tier_every_accepted_payment_traces_to_an_allow_decision`
T4 Govern	The owner, and the Watchdog in a limited way	State changing routes need an owner token (`helm/auth.py`, `require_owner` in `helm/api/routes.py`). The Watchdog can engage the kill switch but has no code path that releases it	`test_unauthenticated_requests_to_state_changing_routes_are_refused`, `test_rt7_tampered_audit_row_is_caught_and_stops_the_executor`, `test_rt9_kill_switch_refuses_a_valid_allow_within_one_request`

Approval levels as implemented. ALLOW is automatic. ESCALATE needs the owner and happens for an amount above the approval limit, a new payee, a quarantined document, a changed bank account for an unprotected payee, an amount over three times the usual, a daily or monthly cap that would be crossed, or a mandate that has ended. DENY is final and happens for the kill switch, a protected payee with changed bank details, a duplicate reference, or a reserve that is too small. HOLD is a delay until the earliest safe date.

The mandate (`daily_cap`, `monthly_cap`, `mandate_expires` on the owner rules) is enforced in `policy.py` and tested in `tests/test_mandate_auth.py`. Statutory and payroll items are exempt from the caps, because they are never delayed, but they do stop being automatic after the mandate ends. Every change to the mandate is written to the audit log as `rules.changed` and is part of the policy hash that is anchored on chain.

2 Threats, controls and tests

ID	Threat	Controls	Tests
TH1	Prompt injection in a document	Reader has no tools and returns typed output. The injection check normalises invisible, full width and soft hyphen characters and HTML comments first. Flagged documents are quarantined and escalate	RT1, RT2, `test_x1_injection_in_a_customer_message...`
TH2	Payee fraud and changed bank details	Protected payees are denied when details change. Other payees escalate. New and look alike payees escalate	RT3, `test_x2_a_look_alike_supplier_name...`, `test_x4_hidden_characters_in_a_bank_account...`
TH3	Excessive agency	Approval limit, daily and monthly caps, mandate expiry, chance constraint on the minimum balance. Only the Policy Engine decides	RT4, RT5, `test_mandate_auth.py`, `test_x3_a_decimal_trick...`
TH4	Replay and duplicates	Duplicate reference rule. Bank keys are scoped to one account, one amount and one payment, expire in five minutes and are single use. Executed proposals are not selected again	RT6
TH5	Tampering with history	Append only triggers, hash chain, Watchdog verification after every run, on-chain anchor of the head hash	RT7, `tests/test_onchain.py`
TH6	Credentials and supply chain	Secrets only in `.env`. Only the Executor mints keys. The anchor key is a throwaway test network key. `pip-audit` and `pnpm audit` outputs are in `docs/evidence/`	tier tests
TH7	Rogue or cascading agent	Typed hand offs. Fail closed engine. Watchdog attack self test after every run. Kill switch	RT9
TH8	Bad inputs and forecast error	Calibrated bands with a stated track record. Conservative chance constraint. Owner approval above the limit	`test_policy_engine.py`
TH9	Data exposure	Only the fields a step needs are passed on. Only hashes go on chain	`test_only_hashes_reach_the_chain`
TH10	Denial of service by large input	Documents are limited to 20,000 characters. Oversized input becomes an inert quarantined record. Owner login is rate limited to five tries a minute	RT10, `test_login_is_rate_limited`

A finding from the red team suite itself was that the first version of the injection detector missed "ignore your previous instructions", because it allowed only one qualifier word. The case is now part of the suite (`test_rt2_hidden_instruction_is_flagged_not_followed`) and the pattern was widened.

3 Dependency audit

`docs/evidence/pip-audit.txt` shows no known vulnerabilities in the API environment (a first run flagged only `pip` itself, which is a tool and not a runtime dependency, and was upgraded). `docs/evidence/pnpm-audit.txt` shows one high finding, `braces`, reached through `eslint-config-next` in the web app. It has no patched version, it is a development dependency, and it is not part of the production bundle. It is accepted and listed as risk R13.

4 Known risk list

ID	Risk	Rating after controls	Next step
R1	Owner login is one shared passcode. In the public demo it is shown on the login dialog on purpose, because the data is synthetic. There are no per user roles	High for a real deployment, accepted for the demo	Replace with a proper identity provider and roles before any pilot
R2	The injection detector is a set of patterns and can be bypassed with new wording	Medium	The real protection is that the Reader has no tools and every payment still passes the Policy Engine. Add a model based check and keep growing the red team suite
R3	Real bank failures such as delays, partial failures and reversals are not exercised, because the bank is a simulation	Medium	Run in shadow mode against a real sandbox, then with a licensed partner
R4	The mandate caps are checked at the payment's own date and do not yet look at the whole plan beyond the same pass	Low	Add a rolling window check
R5	The forecast can be too confident on short histories. On the seeded data the 80 percent range covered 77 percent of outcomes	Medium	Validate on real business data
R6	The self assessment was done by the builders and there was no independent penetration test	Medium	Commission an independent test
R7	The live language model path is tested with recorded real Gemini responses on four documents and with stub tests, but not on a wide range of documents or models	Medium	Record more documents and add a model based injection check
R8	The on-chain anchor depends on a public test network that may be paused or retired	Low	A failed anchor is logged and retried on the next run. A second test network is a settings change
R9	A person with database admin rights can edit rows. The hash chain and the anchor reveal this after the fact but do not prevent it	Medium	Separation of duties and access reviews in production
R10	The anchor runs after every pipeline run when it is configured, and by hand otherwise. Rows written after the latest anchor are protected only by the hash chain	Low	Anchor on a schedule as well
R11	Licensing and data protection duties are not settled for a live service	Medium	Operate through a licensed partner and obtain legal review
R12	The public demo resets itself on a timer and has a known passcode, so it must never hold real data	Accepted	Keep the demo synthetic
R13	`braces` in the web development toolchain has an unpatched denial of service advisory	Low	Move to a toolchain release that no longer depends on it

5 How to reproduce the evidence

```
cd services/api && . .venv/bin/activate && pytest -v         # unit, red team, mandate, login and on-chain tests
python ../../scripts/run_evidence.py                          # drives the running API over HTTP (start the API first)
python ../../scripts/local_chain_demo.py                      # local chain rehearsal with real transaction hashes
python ../../scripts/deploy_anchor.py                         # public test network deployment (needs a funded throwaway
python ../../scripts/record_fx.py                             # real Frankfurter response
```